



CHALLENGE 16 / 20

THE OT LAPTOP ON THE WRONG NETWORK

INDUSTRIAL CONTROL SYSTEM EXPOSURE ASSESSMENT

NATIONAL LEVEL HACKATHON • 2026

POWERED BY SYNTHOQUEST PVT LTD

Student Project Development Challenge • No organizer-provided evidence package.

1 | PROBLEM BACKGROUND

A 1,200 MW thermal power plant found a contractor's engineering laptop connected both to the corporate network and, through a USB adapter, to the control network for six days. The laptop had been used at four other plants. The control system uses proprietary and standard industrial protocols. Controllers keep no command log. The distributed control system holds a limited online-change audit log.

Project files read back from three controllers differ by checksum from the baseline, and the vendor says routine online edits explain this. A setpoint trend shows 0.3% unexplained drift. A safety-system bypass flag toggled twice, and only one toggle is explained.

The plant has no outage window for eleven months. Controllers cannot be probed or power-cycled, and the safety system's integrity must be established before the next demand.

2 | PROBLEM STATEMENT

Develop an **Industrial Command Exposure Assessment System** that determines whether unauthorised commands or logic changes reached controllers or the safety system, which ones, and what process impact followed, using only passive and safety-safe evidence.

The system must separate sanctioned engineering and routine online edits from unauthorised changes, and produce a quantified statement of safety-system integrity that supports a continue-or-stop decision.

Hard constraints

- No active probing is permitted: safety is a hard constraint on the method itself.
- Protocol documentation is partial, so teams must define and justify their own simulated protocol model.
- Controller command logs do not exist and baseline comparison is unreliable.
- Legitimate engineering activity looks like unauthorised change.
- No real industrial equipment may be controlled or tested.

No evidence package or dataset is provided. Teams decide what to build, what data they need, how to simulate the environment and how to validate the result. No tools or architecture are prescribed.

3 | MANDATORY FEATURES

Teams must develop a working software solution. The system must provide the following features. How each is achieved is for the team to decide.

01	Passive Evidence Collection Model	Ability to work only from safe, passive evidence.
02	Command Reach Inference	Ability to infer whether commands reached controllers.
03	Logic-Change Detection	Ability to detect logic or parameter changes without full logs.
04	Sanctioned-Edit Separation	Ability to separate routine edits from unauthorised changes.
05	Process Impact Assessment	Ability to relate changes to process behaviour.
06	Safety-System Integrity Assessment	Ability to state integrity of the safety system with quantified risk.
07	Baseline Unreliability Handling	Ability to reason when the baseline itself is uncertain.
08	Continue-or-Stop Decision Support	Ability to support an operational decision under uncertainty.

The system must clearly distinguish between:

AUTHORISED	UNAUTHORISED	NO EXPOSURE	POSSIBLE EXPOSURE	UNKNOWN
------------	--------------	-------------	-------------------	---------

A lack of abnormal behaviour must not be presented as proof that nothing happened.

Build, don't search. This is a development challenge. Teams must create the system and the controlled environment that proves it works. Submitting an analysis of a given dataset is not a solution.

4 | EXPECTED OUTPUT

Live demonstration. The team must show that its system can handle a complex controlled scenario created by the team. The demonstration must show:

01	Teams' own simulated plant with a documented protocol model
02	Dual-homed engineering laptop with sanctioned and unauthorised activity
03	Passive-only evidence use
04	Detection of unauthorised changes among routine edits
05	Process impact assessment
06	Safety-system integrity statement with quantified risk
07	Continue-or-stop recommendation

The team must be able to explain why the system reached each major conclusion.

Validation standard (every team)

- Build a scenario generator with known ground truth and evaluate on at least ten distinct scenarios, including ones with deliberately misleading evidence.
- At the demonstration, judges change scenario parameters on the spot. The system must adapt without being rebuilt.
- Include an adversary variant that actively tries to defeat the system, and report how the system held up.
- Stated confidence must match measured accuracy. Overconfident or unexplained conclusions lose marks.
- No real systems, real personal data or functional malware may be used.

Submission. Each team must submit:

01	Working Prototype	A functional implementation of the proposed solution.
02	Source Code	Complete source code required to reproduce the project.
03	Technical Architecture	Architecture and component description.
04	Validation Environment	A controlled environment created by the team to demonstrate the solution.
05	Technical Documentation	Implementation, design and testing documentation.
06	Forensic Investigation Report	A professional report generated from the demonstrated investigation.
07	Demonstration	Live technical demonstration to the judging panel.
08	Limitations	Known limitations and situations where the system cannot establish a reliable conclusion.